

CSED 312 Project 3 Final Report

김지현(20220302) 전현빈(20220259)

Design Implementation

1. Frame table

Implementation & Improvement from the previous design

`frame table`은 리스트(`list`)로 구현할 예정이며, 리스트의 각 엔트리에 해당하는 구조체를 생성하기 위해 `vm` 디렉터리 내에 `frame.h` 파일을 작성하였다.

```
./vm/frame.h
struct fte
{
    void *frame_addr;    /* VA: Kernel virtual page. */
    void *page_addr;    /* PA: User virtual page. */
    struct thread *t;    /* 어떤 thread가 이 frame(그의 입장에서는 page겠지만,)을 소유
하나? */
    struct list_elem list_elem; /* 실제 fte가 연결되는 frame table(table은 list로
구현되므로)에 insert*/
};
```

`frame table entry`는 다음과 같은 요소들로 구성된다: 커널 페이지(`frame_addr`), 사용자 페이지(`page_addr`), 해당 엔트리를 소유하는 스레드(`t`), 그리고 리스트 연결을 위한 `list_elem`(프레임 테이블에 연결될 리스트).

또한, `./vm/frame.c` 파일에서 `frame table`과 관련된 변수 및 함수를 선언하였다.

```
static struct list frameTable; /* 프레임 테이블, 실제 fte 소유 주체 */
static struct lock fTableLock; /* frame table에 대한 atomic access를 구현 */
```

먼저, 실제로 `fte`들을 소유하는 `frameTable`을 리스트(`list`) 형태로 선언하였다. 그리고 `frameTable`이 작동할 때 원자성을 보장하기 위해 `fTableLock`이라는 lock을 선언하였다.

```
// ./vm/frame.c
void
init_Lock_and_Table ()
{
    list_init (&frameTable);
    lock_init (&fTableLock);
}

// ./thread/init.c
```

```
int
main (void)
{
    ...
    init_Lock_and_Table();
    ...
}
```

다음으로, `frameTable`을 초기화하는 함수를 구현하였다. `list` 데이터 타입으로 `frameTable`을 선언했으므로, `list_init` 함수를 사용하여 `frameTable`을 초기화하였다. 또한, `fTableLock`이라는 락도 `lock_init` 함수를 통해 초기화하였다.

위에서 설명한 `init_Lock_and_Table` 함수는 `thread/init.c`의 `main` 함수에서 Pintos가 시작될 때 호출되어 초기화된다.

```
// ./userprog/process.c
static bool
setup_stack (void **esp)
{
    uint8_t *frame_addr;
    bool success = false;

    //frame_addr = palloc_get_page (PAL_USER | PAL_ZERO); // Old
    frame_addr = falloc_get_page(PAL_USER | PAL_ZERO, PHYS_BASE - PGSIZE); // New
    if (frame_addr != NULL)
    {
        success = install_page (((uint8_t *) PHYS_BASE) - PGSIZE, frame_addr, true);
        if (success){
            *esp = PHYS_BASE;
        }
        else{
            //palloc_free_page (frame_addr); // Old
            falloc_free_page(frame_addr); // New
        }
    }
    return success;
}
```

`setup_stack` 함수에서 `palloc` 대신 `falloc`을 사용하는 이유는, 메모리 할당 및 관리를 더욱 체계적으로 처리하기 위해서이다. `palloc`은 단순히 페이지를 할당하는 역할만 수행하지만, `falloc`은 추가적으로 할당된 페이지와 관련된 메타데이터를 `frameTable`에 저장하고 관리할 수 있다. 이를 통해, 메모리 해제 시 관련 정보를 효율적으로 추적하고, 여러 스레드가 동시에 접근할 때도 동기화를 보장할 수 있다. 따라서 `falloc`을 사용함으로써 메모리 관리의 확장성과 안정성을 확보할 수 있다.

```
void *
falloc_get_page(enum palloc_flags flags, void *page_addr)
{
    struct fte *e;
```

```

void *frame_addr;
lock_acquire (&fTableLock);
frame_addr = palloc_get_page (flags);
if (frame_addr == NULL)
{
    // swapping 구현
    frame_addr = palloc_get_page (flags);
    if (frame_addr == NULL)
        return NULL;
}

e = (struct fte *)malloc (sizeof *e);
e->frame_addr = frame_addr;
e->page_addr = page_addr;
e->t = thread_current ();

list_push_back (&frameTable, &e->list_elem);

lock_release (&fTableLock);
return frame_addr;
}

```

먼저, `falloc`은 `palloc`을 사용해 `upage`에 매핑될 `kpage`를 할당받는다. 이후, `frame table entry`를 `malloc`을 사용해 동적으로 할당한 뒤, 앞서 설명한 `fte`의 각 요소를 초기화한다. 초기화된 `fte`는 `frameTable`에 `list_push_back`을 이용해 추가된다. 이 과정에서 `frameTable`에 여러 스레드가 동시에 접근하지 않도록 `fTableLock`을 사용해 원자성을 보장하였다. 만약 `palloc`이 실패하면, 락을 해제(`lock_release`)한 후 `NULL`을 반환한다. 성공 시에는 새롭게 할당된 `kpage`를 반환한다.

```

void
falloc_free_page (void *frame_addr)
{
    struct fte *e;
    lock_acquire (&fTableLock);
    e = getFte (frame_addr);
    if (e == NULL)
        sys_exit (-1);
    do_free_frame(e);

    lock_release (&fTableLock);
}

void do_free_frame(struct fte *targetFTE)
{
    list_remove(&targetFTE -> list_elem);
    palloc_free_page(targetFTE -> frame_addr);
    pagedir_clear_page(targetFTE -> t -> pagedir, targetFTE -> page_addr);
    free(targetFTE);
}

```

`setup_stack` 함수에서 `install_page`가 실패할 경우, `falloc`으로 할당한 `kpage`를 해제해야 한다. 이를 위해 먼저, `kpage`에 해당하는 `frame table entry`를 찾기 위해 `getFte` 함수를 호출한다. 이후, 찾은 `frame table entry`를 `list_elem`에서 제거(`list_remove`)한다. 그리고 `palloc_free_page` 함수를 사용해 해당 `kpage`를 해제하며, 마지막으로 `pagedir_clear_page`를 호출해 `kpage`에서 `upage`로의 접근을 차단한다. 최종적으로, `fte`를 해제하여 메모리 누수를 방지한다. 이 과정에서도 `fTableLock`을 사용해 `frameTable`에 대한 원자성을 유지하였다.

```
struct fte *
getFte (void* frame_addr)
{
    struct list_elem *e;
    for (e = list_begin (&frameTable); e != list_end (&frameTable); e = list_next
(e))
        if (list_entry (e, struct fte, list_elem)->frame_addr == frame_addr)
            return list_entry (e, struct fte, list_elem);
    return NULL;
}
```

`falloc_free_page`에서 사용하는 `get_fte` 함수는 특정 `kpage`에 대응되는 `frame table entry`를 반환한다. 이 함수는 `for` 루프를 통해 `frameTable`의 각 엔트리를 확인하며, 대응되는 `kpage`를 찾으면 해당 `list_entry`를 반환한다. 만약 대응되는 `kpage`가 없을 경우, `NULL`을 반환하도록 구현하였다.

Difference from design report

디자인 리포트에서 작성한 pseudocode 를 바탕으로 작성하였다.

3. Supplemental page table

lazy loading에 supplemental page table이 사용되어서, supplemental page table 을 먼저 설명할 예정이다.

Implementation & Improvement from the previous design

```
// ./vm/page.h

struct spte
{
    void *frame_addr;
    void *page_addr;
    struct hash_elem hash_elem;

    int status;

    struct file *file;
    off_t ofs;
    uint32_t read_bytes, zero_bytes;
    bool isWritable;
    int swap_id;
};
```

Supplemental Page Table은 해시 테이블(hash table)을 이용해 구현하는 것이 권장되므로, 위와 같이 선언하였다. 해당 구조체는 Supplemental Page Table Entry를 나타내며, 다음과 같은 요소들로 구성된다: 물리 페이지(physical page), 가상 페이지(virtual page), 해시 요소(hash element), 상태(status)로 구성되며, 상태는 PAGE_FRAME, PAGE_ZERO, PAGE_SWAP, 또는 PAGE_FILE 중 하나로 설정된다.

또한, 페이지에 파일이 연결되어 있는 경우를 위해, 파일 포인터(file pointer), 오프셋(offset), 읽어야 하는 바이트 수, 0으로 설정해야 할 바이트 수, 그리고 해당 페이지의 쓰기 가능 여부를 저장한다.

```
// ./threads/thread.h

struct thread
{
    ...
    struct hash spt;
};

// ./threads/thread.c
tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
    ...
    init_spt(&t->spt);
    ...
}
```

추가적으로, thread 구조체에 hash 타입의 Supplemental Page Table을 추가하였다. thread_create 함수에서 Supplemental Page Table을 초기화하며, init_spt 함수는 ./vm/page.c 파일에서 더 자세히 설명될 예정이다.

```
static hash_hash_func spt_hash_func;
static hash_less_func comp_spt_va;
```

./vm/page.c 파일에서는 Supplemental Page Table과 관련된 함수 및 변수를 구현하였다. 먼저, hash_init 함수를 사용하여 Supplemental Page Table을 초기화한다. 이 과정에서 해시 함수(hash_hash_func)와 비교 함수(hash_less_func)가 필요하므로, 이를 별도로 선언하였다.

```
static unsigned
spt_hash_func (const struct hash_elem *elem, void *aux)
{
    struct spte *p = hash_entry(elem, struct spte, hash_elem);

    return hash_bytes (&p->page_addr, sizeof (p->frame_addr));
}
```

`hash_hash_func` 함수는 입력으로 전달받은 `hash_elem`에 해당하는 해시 엔트리를 가져와, 이를 기반으로 해시 값을 생성하는 역할을 한다.

```
static bool
comp_spt_va (const struct hash_elem *e1, const struct hash_elem *e2, void *aux)
{
    return hash_entry (e1, struct spte, hash_elem)->page_addr < hash_entry (e2,
    struct spte, hash_elem)->page_addr;
}
```

`comp_spt_va` 함수는 해시 테이블 엔트리를 비교하는 boolean 함수로, 두 엔트리의 가상 주소를 비교하여 true 또는 false를 반환한다.

```
void
init_spt (struct hash *spt)
{
    hash_init (spt, spt_hash_func, comp_spt_va, NULL);
}
```

위에서 설명한 두 함수(`spt_hash_func`와 `comp_spt_va`)를 사용하여 `hash_init` 함수를 호출하고, 이를 통해 `init_spt` 함수를 구현하였다.

```
static void spte_free (struct hash_elem *elem, void *aux);

static void
spte_free (struct hash_elem *e, void *aux)
{
    free(hash_entry (e, struct spte, hash_elem));
}
```

Supplemental Page Table을 삭제하기 위해 `hash_destroy` 함수를 호출해야 한다. 이 과정에서 각 엔트리를 해제하기 위해 `spte_free` 함수가 필요하다. `spte_free` 함수는 전달받은 `elem`에 해당하는 해시 엔트리를 찾아내고, 이를 `free` 함수를 사용해 메모리에서 해제한다.

```
void
destroy_spt (struct hash *spt)
{
    hash_destroy (spt, spte_free);
}
```

`spte_free`를 인자로 넘겨 `hash_destroy`를 호출함으로써, Supplemental Page Table을 제거하는 함수를 구현하였다.

```
#define PAGE_ZERO 0
#define PAGE_FRAME 1
#define PAGE_FILE 2
#define PAGE_SWAP 3
```

Supplemental Page Table 구현의 일환으로, 다음 네 가지 상태를 매크로로 정의하였다:

- **PAGE_ZERO**: 페이지가 아직 할당되지 않은 상태를 나타낸다.
- **PAGE_FRAME**: 페이지가 물리 메모리(**physical memory**)에 매핑된 상태를 나타낸다.
- **PAGE_FILE**: 페이지가 파일 시스템에 저장되어 있고, 필요 시 파일 시스템에서 읽어와야 하는 상태를 나타낸다. 이는 아래에서 설명할 **Lazy Loading**에 사용된다.
- **PAGE_SWAP**: 페이지가 스왑 공간에 저장된 상태를 나타내며, 이에 대한 자세한 내용은 아래 **Swap Table** 설명에서 다룰 예정이다.

각 상태에 맞게 **Supplemental Page Table Entry**를 초기화하는 함수를 작성하였으며, 위에서 정의한 매크로의 순서대로 설명을 진행한다.

```
void
init_spte_zero (struct hash *spt, void *page_addr)
{
    struct spte *e;
    e = (struct spte *) malloc (sizeof *e);

    e->page_addr = page_addr;
    e->frame_addr = NULL;

    e->status = PAGE_ZERO;

    e->file = NULL;
    e->isWritable = true;

    hash_insert (spt, &e->hash_elem);
}
```

PAGE_ZERO의 경우, 데이터가 모두 0으로 채워져야 한다. 이 상태는 아직 **page_addr**가 매핑되지 않은 상태이므로, **frame_addr**를 **NULL**로 설정한다. 또한, **page_addr**는 함수 인자로 전달받은 값(**page_addr**)을 할당하며, **file**은 **NULL**, **isWritable**은 **true**로 설정한다. 마지막으로, **hash_insert** 함수를 사용해 해당 엔트리를 **Supplemental Page Table**에 추가한다.

```
void
init_spte_frame (struct hash *spt, void *page_addr, void *frame_addr)
{
    struct spte *e;
    e = (struct spte *) malloc (sizeof *e);

    e->page_addr = page_addr;
    e->frame_addr = frame_addr;
```

```

    e->status = PAGE_FRAME;

    e->file = NULL;
    e->isWritable = true;

    hash_insert (spt, &e->hash_elem);
}

```

PAGE_FRAME의 경우 **PAGE_ZERO**와 유사하지만, **frame_addr**를 함수 인자(frame_addr)로 받아 이를 할당하는 과정을 추가하였다.

```

struct spte *
init_spte_file(struct hash *s, void *_p, struct file *_f, off_t _o, uint32_t _r,
uint32_t _z, bool _i)
{
    struct spte *e;

    e = (struct spte *)malloc (sizeof *e);

    e->page_addr = _p;
    e->frame_addr = NULL;
    e->file = _f;
    e->ofs = _o;
    e->read_bytes = _r;
    e->zero_bytes = _z;
    e->isWritable = _i;

    e->status = PAGE_FILE;

    hash_insert (s, &e->hash_elem);

    return e;
}

```

PAGE_FILE의 경우, 파일 참조를 위해 필요한 값들을 함수 인자로 받아 각각 할당한다. 해당 값에는 **file**, **offset**, **_read_bytes**, **_zero_bytes**, **isWritable** 등이 포함된다. 이 상태에서도 **frame_addr**는 매핑되지 않은 상태이므로, **NULL**로 설정한다.

PAGE_SWAP 상태는 6번에서 설명할 스왑 과정을 통해 다루도록 하겠다.

```

void
delete_and_free (struct hash *spt, struct spte *spte)
{
    hash_delete (spt, &spte->hash_elem);
    free (spte);
}

```


`delete_and_free` 함수는 특정 엔트리를 `Supplemental Page Table`에서 삭제하고 메모리를 해제하는 기능을 한다. 함수는 해당 엔트리의 해시 값을 기반으로 `hash_delete`를 호출하여 `Supplemental Page Table`에서 엔트리를 삭제한 뒤, 삭제된 엔트리를 `free`를 통해 메모리에서 해제한다.

위 과정을 통해 `Supplemental Page Table`과 관련된 모든 함수, 그리고 엔트리 관련 기능들을 구현하였다.

```
// ./userprog/process.c

static bool
setup_stack (void **esp)
{
    ...
    success = install_page (((uint8_t *) PHYS_BASE) - PGSIZE, frame_addr, true);
    if (success){
        init_frame_spte(&thread_current()->spt, PHYS_BASE - PGSIZE, frame_addr);
// 이 부분
        *esp = PHYS_BASE;
    }
    ...
}
```

`setup_stack` 함수에서 `install_page`가 성공하면, `init_frame_spte` 함수를 호출하여 할당된 `kpage`를 `Supplemental Page Table`에 등록한다.

Difference from design report

디자인 레포트 초안에서는 4가지 상태(`PAGE_ZERO`, `PAGE_FRAME`, `PAGE_FILE`, `PAGE_SWAP`)를 고려하지 못했기 때문에, 초기화 과정을 단일한 방식으로만 구상하였다. 이후, 상태별로 초기화 과정을 각각 설계하여 보완하였다. (아래의 구현에서 사용하기 위함) 또한, 해시 테이블 내부 함수 활용이 미흡하여 `Supplemental Page Table`을 초기화하고 삭제하는 함수의 구현이 누락되었으나, 이를 보완하며 관련 함수들을 추가로 구현하였다.

2. Lazy loading

Implementation & Improvement from the previous design

```
// ./userprog/process.c

static bool
load_segment (struct file *file, off_t ofs, uint8_t *upage,
              uint32_t read_bytes, uint32_t zero_bytes, bool writable)
{
    ...
    init_file_spte (&thread_current()->spt, page_addr, file, ofs, page_read_bytes,
page_zero_bytes, writable);
    ...
}
```

기존의 `load_segment` 함수는 파일을 직접 메모리에 로드하였다. 그러나, 이제는 Lazy Loading 과정을 구현하기 위해 `init_file_spte` 함수를 호출하도록 수정하였다. 이를 통해 page fault가 발생하면 해당

Supplemental Page Table Entry의 요소(즉, 파일 정보)를 참조하여 필요한 페이지를 메모리에 로드할 수 있도록 처리하였다.

```
static void
page_fault (struct intr_frame *f)
{
    ...
    page_addr = pg_round_down (fault_addr);

    if (is_kernel_vaddr (fault_addr) || !not_present) {sys_exit (-1);}

    spt = &thread_current()->spt;

    spe = get_spte(spt, page_addr);

    if(user == true){
        esp = f -> esp;
    }else{
        thread_current()->esp;
    }

    if (load_page (spt, page_addr)) {
        return;
    }
    ...
}
```

페이지 폴트 발생 시 Lazy Loading이 실행될 수 있도록 page fault handler에서 load_page 함수를 호출하였다. load_page의 실행 과정은 다음과 같다:

```
extern struct lock FileLock;

bool
load_page (struct hash *spt, void *page_addr)
{
    struct spte *e;
    uint32_t *pagedir;
    void *frame_addr;

    e = get_spte (spt, page_addr);
    if (e == NULL){ sys_exit (-1); }

    frame_addr = falloc_get_page (PAL_USER, page_addr);
    if (frame_addr == NULL){ sys_exit (-1); }

    bool was_holding_lock = lock_held_by_current_thread (&FileLock);

    prepare_mem_page(e, frame_addr, was_holding_lock);

    pagedir = thread_current ()->pagedir;
```

```

    if (!pagedir_set_page (pagedir, page_addr, frame_addr, e->isWritable))
    {
        falloc_free_page (frame_addr);
        sys_exit (-1);
    }

    e->frame_addr = frame_addr;
    e->status = PAGE_FRAME;

    return true;
}

void prepare_mem_page(struct spte *spte, void *frame_addr, bool flag)
{
    switch (spte->status)
    {
    case PAGE_ZERO:
        memset (frame_addr, 0, PGSIZE);
        break;
    case PAGE_SWAP:
        swap_in(spte, frame_addr);
        break;
    case PAGE_FILE:
        if (!flag)
            lock_acquire (&FileLock);

        if (file_read_at (spte->file, frame_addr, spte->read_bytes, spte->ofs) !=
spte->read_bytes)
        {
            falloc_free_page (frame_addr);
            lock_release (&FileLock);
            sys_exit (-1);
        }
        memset (frame_addr + spte->read_bytes, 0, spte->zero_bytes);
        if (!flag)
            lock_release (&FileLock);

        break;

    default:
        sys_exit (-1);
    }
}

```

1. **kpage** 할당 페이지 폴트가 발생하면 **falloc**을 사용해 새로운 **kpage**(kernel page)를 할당한다.
2. 메모리 초기화 (**prepare_mem_page**) **switch-case** 문을 사용하여 각 상태(**PAGE_ZERO**, **PAGE_SWAP**, **PAGE_FILE**)에 따라 처리한다:
 - **PAGE_ZERO**: **memset** 함수를 사용해 메모리를 0으로 초기화한다.

- **PAGE_SWAP**: 스왑 테이블 과정을 통해 처리하며, 이에 대한 상세 내용은 아래(6번)에서 설명한다.
- **PAGE_FILE**:
 - **file_read_at** 함수를 통해 파일에서 데이터를 읽어온다.
 - 남은 메모리 영역은 **memset**을 사용해 0으로 초기화한다.
 - 여러 프로세스가 동일 파일에 접근하는 것을 방지하기 위해 **FileLock**을 사용하여 원자적으로 처리한다.

3. 페이지 디렉터리 및 **Supplemental Page Table** 업데이트 새롭게 로드한 데이터를 기반으로 페이지 디렉터리를 설정하고, 해당 **Supplemental Page Table Entry**를 업데이트한다.

```
struct spte *
get_spte (struct hash *spt, void *page_addr)
{
    struct spte e;
    struct hash_elem *elem;

    e.page_addr = page_addr;
    elem = hash_find (spt, &e.hash_elem);

    return elem != NULL ? hash_entry (elem, struct spte, hash_elem) : NULL;
}
```

페이지 폴트 처리 과정에서 **get_spte** 함수를 사용하여 **page_addr**에 해당하는 **Supplemental Page Table Entry**를 가져온다. **get_spte** 함수는 **page_addr**를 입력받아 이를 기반으로 해시 테이블에서 적절한 엔트리를 반환한다.

Difference from design report

초기 설계에서는 Lazy Loading 구현을 위해 **page_table**이라는 별도 구조체를 사용하려 하였으나, 이후 **Supplemental Page Table**을 이용한 방식으로 변경하였다. 또한, 디자인 레포트에서는 **PAGE_FILE**과 **PAGE_SWAP**에 대해서만 구상하였으나, 구현 과정에서 **PAGE_ZERO**에 대한 처리도 추가하였다.

4. Stack growth

Implementation & Improvement from the previous design

```
struct thread
{
    ...
    void *esp;
    ...
};
```

Stack Growth를 구현하기 위해 현재 스택 포인터 위치를 추적해야 하므로, **thread** 구조체에 **esp** 포인터를 추가하였다.

```

static void
page_fault (struct intr_frame *f)
{
    ...
    page_addr = pg_round_down (fault_addr);

    if (is_kernel_vaddr (fault_addr) || !not_present) {sys_exit (-1);}

    spt = &thread_current()->spt;

    spe = get_spte(spt, page_addr);

    if(user == true){
        esp = f -> esp;
    }else{
        thread_current()->esp;
    }

    bool isValidExtend = esp - STACK_BUFFER <= fault_addr && STACK_LIMIT <=
    fault_addr;
    if (isValidExtend) {
        init_zero_spte(spt, page_addr);
    }
    ...
}

```

Stack Growth는 페이지 폴트가 발생했을 때 실행된다. 먼저, `pg_round_down` 함수를 사용하여 현재 주소를 페이지 크기의 배수로 내림하여 해당 주소가 속한 페이지의 시작 주소를 계산하고, 이를 `page_addr`에 할당한다. 그리고, `if` 문을 사용하여 스택 확장이 가능한지 조건을 확인한다. 마지막으로, `init_zero_spte` 함수를 사용하여 새로운 **Supplemental Page Table Entry**를 생성한 후, 이를 **Supplemental Page Table**에 추가하였다.

Difference from design report

기존 디자인과 비교하여 다음 두 가지 차별점이 있다. 첫번째로, 초기 설계에서는 스택 성장이 가능한지 확인하는 별도의 함수를 구현하려고 했으나, `if` 문으로 간단히 확인할 수 있음을 고려하여 `page_fault` 함수 내부에서 바로 조건을 확인하도록 수정하였다. 다음으로, 스택 메모리를 확장해주는 함수는 데이터를 0으로 초기화하는 `init_zero_spte` 함수를 사용하도록 변경하였다. 이를 통해 Supplemental Page Table을 구현하는 방식과의 일관성을 유지하였다.

5. File memory mapping

Implementation & Improvement from the previous design

```

// ./threads/thread.h

struct mmf
{
    int id;                                /* 이름 */

```

```

    struct file* file;                /* 무슨 파일에 해당하는지? */
    struct list_elem mmf_list_elem;   /* 마스터 스레드가 갖는 mmf리스트에 끼울 elem */
    void *page_addr;                 /* mapping된 VA */
};

```

새로운 **memory mapped file**(mmf) 구조체를 정의하였다. 이 구조체는 고유 식별자인 **id**, 파일을 참조하는 **file**, 각 스레드의 **mmf_list**에 저장되는 리스트 요소인 **mmf_list_elem**, 그리고 사용자 페이지 주소인 **page_addr**를 포함한다.

```

struct thread
{
    ...
    struct list mmf_list; // 슬레이브 mmf 리스트
    int t_mmf_id;         // 스레드 mmf id
};

```

이 구조체는 스레드 구조체의 요소로 추가되었으며, 각 스레드는 현재 보유한 **mmf**의 수를 나타내는 **mapid**와 이를 리스트 형태로 저장하는 **mmf_list**를 가진다.

```

// ./threads/thread.c
tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
    ...
    thread_mmf_init(t);
    ...
}

void thread_mmf_init(struct thread *t)
{
    list_init (&t->mmf_list);
    t->t_mmf_id = 0;
}

```

thread_create 함수에서 **thread_mmf_init** 함수를 호출하였고 해당 함수에서 **list_init**을 이용해 **mmf_list**를 초기화하고, **mapid**를 0으로 설정하여 mmf를 초기화하였다.

```

// ./threads/thread.c

struct mmf *init_mmf (int id, struct file *file, void *page_addr)
{
    struct mmf *mmf = (struct mmf *) malloc (sizeof *mmf);

    mmf->id = id;
    mmf->file = file;
    mmf->page_addr = page_addr;
}

```

```

uint64_t ofs;
int size = file_length (file);
struct hash *spt = &thread_current ()->spt;

for (ofs = 0; ofs < size; ofs += PGSIZE)
  if (get_spte (spt, page_addr + ofs))
    return NULL;

for (ofs = 0; ofs < size; ofs += PGSIZE)
{
  uint32_t read_bytes = ofs + PGSIZE < size ? PGSIZE : size - ofs;
  init_file_spte (spt, page_addr, file, ofs, read_bytes, PGSIZE - read_bytes,
true);
  page_addr += PGSIZE;
}

list_push_back (&thread_current ()->mmf_list, &mmf->mmf_list_elem);

return mmf;
}

```

`mmf` 초기화를 위해 `init_mmf` 함수를 구현하였다. 이 함수는 `malloc`을 통해 `mmf`를 동적으로 할당한 후, 인자로 전달받은 `id`, `file`, `page_addr`를 초기화한다. 첫 번째 `for` 루프를 사용해 파일이 이미 매핑되어 있는지 확인하며, 중복 매핑이 발견되면 `NULL`을 반환한다. 두 번째 `for` 루프에서는 파일의 각 페이지를 `Supplemental Page Table`에 추가한다. 이때 `init_file_spte`를 호출해 페이지를 초기화하고 `Supplemental Page Table`에 등록한다. 마지막으로 생성된 `mmf`를 스레드의 `mmf_list`에 추가한다.

```

// ./userprog/syscall.c

static void
syscall_handler(struct intr_frame *f)
{
  if(verify_mem_address(f->esp)){
    int argv[3];
    switch (*(uint32_t *) (f->esp))
    {
      ...
      case SYS_MMAP:
        getArgs(f->esp + 4, &argv[0], 2);
        f->eax = sys_mmap((int) argv[0], (void *) argv[1]);
        break;
      case SYS_MUNMAP:
        getArgs(f->esp + 4, &argv[0], 1);
        sys_munmap((int) argv[0]);
        break;
      ...
    }
  }
}

```

`syscall_handler`에서 파일 매핑과 해제를 위한 시스템 호출을 switch case 를 추가하여 구현하였다.

```
// ./userprog/syscall.c

int
sys_mmap(int fd, void *addr) {
    struct thread *t = thread_current();
    //struct file *f = t->pcb->fd_table[fd];
    struct file *f = t->fileTable[fd];
    struct file *opened_f;
    struct mmf *mmf;

    if (f == NULL)
        return -1;

    if (addr == NULL || (int) addr % PGSIZE != 0)
        return -1;

    lock_acquire(&FileLock);

    opened_f = file_reopen(f);
    if (opened_f == NULL) {
        lock_release(&FileLock);
        return -1;
    }

    mmf = init_mmf(t->t_mmf_id++, opened_f, addr);
    if (mmf == NULL) {
        lock_release(&FileLock);
        return -1;
    }

    lock_release(&FileLock);

    return mmf->id;
}
```

`sys_mmap` 함수는 파일 디스크립터(`fd`)와 주소(`addr`)를 인자로 받아 파일을 메모리에 매핑한다. 이 과정에서 `FileLock`을 사용해 atomicity를 보장하고, `file_reopen`을 통해 파일 복사본을 생성한다. 이후, `init_mmf`를 호출하여 `mmf`를 초기화하고, 파일과 주소를 매핑한다. 매핑 후, 스레드의 `mapid`를 1 증가시킨다.

```
// ./userprog/syscall.c

int
sys_munmap(int t_mmf_id) {
    struct thread *t = thread_current();
    struct list_elem *e;
    struct mmf *mmf;
    void *page_addr;

    if (t_mmf_id >= t->t_mmf_id)
        return;
}
```



```

    for (e = list_begin(&t->mmf_list); e != list_end(&t->mmf_list); e =
list_next(e)) {
        mmf = list_entry(e, struct mmf, mmf_list_elem);
        if (mmf->id == t_mmf_id)
            break;
    }
    if (e == list_end(&t->mmf_list))
        return;

    page_addr = mmf->page_addr;

    lock_acquire(&FileLock);

    off_t ofs;
    for (ofs = 0; ofs < file_length(mmf->file); ofs += PGSIZE) {
        struct spte *entry = get_spte(&t->spt, page_addr);
        if (pagedir_is_dirty(t->pagedir, page_addr)) {
            void *frame_addr = pagedir_get_page(t->pagedir, page_addr);
            file_write_at(entry->file, frame_addr, entry->read_bytes, entry->ofs);
        }
        delete_and_free(&t->spt, entry);
        page_addr += PGSIZE;
    }
    list_remove(e);

    lock_release(&FileLock);
}

```

`sys_munmap` 함수는 인자로 전달받은 `mapid`에 해당하는 `mmf`를 찾고, 각 페이지의 `spte`를 가져온다. 페이지가 수정된 경우, `pagedir_is_dirty`와 `file_write_at`을 사용해 변경 사항을 파일에 기록한다. 이후, `page_delete`로 Supplemental Page Table에서 해당 엔트리를 삭제하고, `list_remove`를 통해 `mmf_list`에서도 제거한다. 이 과정 역시 `FileLock`을 사용하여 atomicity를 유지한다.

Difference from design report

기존 디자인 레포트에서는 `mmf` 구조체에 `page_addr`를 포함하지 않았으나, 파일이 어떤 물리 메모리에 매핑되는지를 추적하기 위해 `page_addr` 요소를 추가하였다. 또한, 파일 매핑과 해제를 처리하기 위해 단순한 구상만 제시되었던 시스템 호출에, 이를 구체적으로 구현하는 `init_mmf` 함수를 추가하였다. 이를 통해 Lazy Loading과 파일 접근의 안정성을 보장하고, `PAGE_FILE` 및 `PAGE_SWAP`뿐만 아니라 `PAGE_ZERO`에 대한 처리도 포함하여 구현을 완성하였다.

6. Swap table

Implementation & Improvement from the previous design

```

// ./vm/swap.c

static struct bitmap *swap_valid_table;
static struct block *swap_disk;

```

`swap_valid_table`을 비트맵 형식으로 선언하여 스왑할 공간을 효율적으로 관리할 수 있도록 하였다. 비트맵 값이 0이면 해당 스왑 슬롯이 사용 가능(`swap in` 가능), 1이면 사용 중(`swap out` 가능)을 의미하도록 구현하였다. 또한, 스왑 영역에서 데이터를 읽고 쓰는 과정이 필요하므로, 블록 디바이스를 나타내는 `swap_disk` 포인터를 추가하였다.

```
// ./vm/swap.c

#define SECTOR_NUM (PGSIZE / BLOCK_SECTOR_SIZE)

void init_swap_valid_table()
{
    swap_disk = block_get_role(BLOCK_SWAP);
    swap_valid_table = bitmap_create(block_size(swap_disk) / SECTOR_NUM);

    bitmap_set_all(swap_valid_table, true);
    lock_init(&swapLock);
}
```

스왑 테이블을 활성화하기 위해 `init_swap_valid_table` 함수를 구현하였다. 이 함수는 `block_get_role`을 호출하여 스왑 영역으로 사용할 블록 디바이스를 가져오고 이를 `swap_disk` 변수에 저장한다. 이후, `bitmap_create`를 사용하여 `swap_valid_table`을 생성하며, 비트맵 크기는 스왑 디스크 크기를 `SECTOR_NUM`으로 나눈 값으로 설정하였다. 마지막으로, `bitmap_set_all` 함수를 호출하여 비트맵 값을 모두 `true`로 설정, 초기 상태를 `swap out` 가능으로 표시하였다. 해당 초기화 함수는 `main` 함수에서 호출되어 실행된다.

```
// ./thread/init.c

int
main (void)
{
    ...
    init_swap_valid_table();
    ...
}
```

그리고 `main` 함수에서 `init_swap_valid_table` 함수를 호출하여 초기화해주었다.

```
// ./vm/swap.c

static struct lock swap_lock;

void init_swap_valid_table()
{
    ...
    lock_init(&swapLock);
}
```

스왑 과정을 구현하기 위해 `swap_lock`을 선언하여 스왑 작업이 atomic 하게 수행되도록 하였다.

```
// ./vm/swap.c

void swap_in(struct spte *page, void *kva)
{
    int i;
    int id = page->swap_id;

    lock_acquire(&swapLock);
    {
        if (id > bitmap_size(swap_valid_table) || id < 0)
        {
            sys_exit(-1);
        }

        if (bitmap_test(swap_valid_table, id) == true)
        {
            sys_exit(-1);
        }

        bitmap_set(swap_valid_table, id, true);
    }

    lock_release(&swapLock);

    for (i = 0; i < SECTOR_NUM; i++)
    {
        block_read(swap_disk, id * SECTOR_NUM + i, kva + (i * BLOCK_SECTOR_SIZE));
    }
}
```

`swap_in` 함수는 스왑 슬롯을 스왑 인 가능한 상태로 전환한다. 이 함수는 먼저 `swapLock`을 통해 동기화를 보장하며, 스왑 ID가 비트맵 범위 내에 유효한지 확인한다. 이후, `bitmap_test`를 호출하여 해당 ID가 참조하는 스왑 슬롯이 사용 중인지 검사한다. 반환값이 `true`라면 해당 슬롯이 비어 있음을 나타낸다. 그런 다음, `bitmap_set`을 사용하여 해당 슬롯을 `true`로 설정하여 사용 중임을 표시한다. 작업이 완료되면 락을 해제 (`lock_release`)한다. 그리고 `block_read` 함수를 통해 `swap_disk` 안을 읽어온다.

```
bool
load_page (struct hash *spt, void *upage)
{
    ...
    switch (e->status)
    {
        case PAGE_ZERO:
            memset (kpage, 0, PGSIZE);
            break;

        case PAGE_SWAP:
            swap_in(e, kpage);
    }
}
```

```

        break;
    ...
}

```

PAGE_SWAP 상태 처리에서 **swap_in** 함수를 호출하여 스왑 작업을 구현하였다. 이는 앞서 설명한 **load_page** 함수에서 페이지가 **PAGE_SWAP** 상태일 때 실행된다.

```

// ./vm/swap.c

int swap_out(void *addr)
{
    int tmp;

    lock_acquire(&swapLock);
    tmp = bitmap_scan_and_flip(swapTable, 0, 1, true);
    lock_release(&swapLock);

    int i = 0;
    while (i < SECTOR_NUM) {
        block_write(swapDisk, tmp * SECTOR_NUM + i, addr + (BLOCK_SECTOR_SIZE *
i));
        i++;
    }

    return tmp;
}

```

swap_out 함수는 스왑 영역에 페이지를 저장하는 역할을 수행한다. 먼저, **bitmap_scan_and_flip**을 호출하여 비어 있는 스왑 슬롯을 찾고 해당 슬롯의 상태를 **true**로 변경한다. 이 과정은 **swapLock**을 사용해 atomic 하게 실행된다. 이후, 해당 스왑 슬롯에 해당하는 블록 디바이스에 **block_write**를 사용하여 데이터를 저장한다. 데이터를 저장하는 작업은 **for** 루프를 통해 구현되며, 페이지 데이터를 블록 단위로 스왑 슬롯에 기록한다.

```

./vm/frame.c

static struct fte *clock_cursor;

void
frame_init ()
{
    ...
    clock_cursor = NULL;
}

void *
falloc_get_page(enum palloc_flags flags, void *upage)
{
    ...
    if (frame_addr == NULL)
    {

```

```

    evict_page();
    ...
}
...
}

```

`clock_cursor`는 프레임 테이블 내 교체 대상 페이지를 탐색하기 위해 사용되는 엔트리이다. 물리 메모리에 여유 공간이 없을 경우, `evict_page` 함수에서 `clock_cursor`를 활용하여 교체 대상 프레임을 찾는다.

`falloc_get_page` 함수는 필요 시 `evict_page`를 호출하여 새로운 페이지를 할당하기 위한 공간을 확보한다.

```

void evict_page() {
    ASSERT(lock_held_by_current_thread(&fTableLock));

    struct fte *e = clock_cursor;
    struct spte *s;

    do {
        if (e != NULL) {
            pagedir_set_accessed(e->t->pagedir, e->page_addr, false);
        }

        if (clock_cursor == NULL || list_next(&clock_cursor->list_elem) ==
list_end(&frameTable)) {
            e = list_entry(list_begin(&frameTable), struct fte, list_elem);
        } else {
            e = list_next(e);
        }
    } while (!pagedir_is_accessed(e->t->pagedir, e->page_addr));

    s = get_spte(&thread_current()->spt, e->page_addr);
    s->status = PAGE_SWAP;
    s->swap_id = swap_out(e->frame_addr);

    lock_release(&fTableLock); {
        falloc_free_page(e->frame_addr);
    } lock_acquire(&fTableLock);
}

```

`evict_page` 함수는 물리 메모리가 부족할 때 사용되지 않는 페이지를 스왑 영역으로 내보내는 기능을 담당한다. 이 함수는 시계 알고리즘(Clock Algorithm)을 사용하여 교체 대상 페이지를 선택한다. 먼저, `frame_lock`을 호출한 스레드가 락을 보유하고 있는지 확인한 후, `clock_cursor`를 기준으로 탐색을 시작한다.

`pagedir_set_accessed`를 호출하여 현재 페이지의 `accessed` 비트를 `false`로 초기화한다. 그리고, `pagedir_is_accessed`를 사용하여 현재 페이지가 최근에 접근되었는지 확인한다. 최근에 접근되지 않은 페이지를 찾을 때까지 `clock_cursor`를 이동하며 프레임 테이블을 탐색한다. 프레임 테이블 끝에 도달하면 다시 처음부터 탐색을 이어간다. 그 다음, 교체 대상이 되는 페이지의 페이지 테이블 엔트리(`spte`)를 가져온다. 해당 페이지의 상태를 `PAGE_SWAP`으로 설정하고, `swap_out`을 호출하여 페이지 내용을 스왑 영역에 저장한다. 반환된 `swap_id`는 스왑 슬롯의 저장 위치를 추적하기 위해 사용된다. 마지막으로, 페이지가 스왑 영역으로 저장된 후, 락을 해제(`lock_release`)하고 `falloc_free_page`를 호출하여 해당 프레임을 physical 메모리에서 해제한다.

Difference from design report

디자인 레포트와 비슷한 형식으로 구현하였다.

7. On process termination

Implementation & Improvement from the previous design

```
./userprog/process.c
void
process_exit (void)
{
  ...
  for (i = 0; i < cur->mapid; i++){
    sys_munmap (i);
  }
  destroy_spt (&cur->spt);
  ...
}
```

프로세스가 종료될 때, 모든 mmf를 닫아야 하므로 해당 프로세스가 보유한 모든 mapid에 대해 sys_munmap 함수를 호출하여 매핑을 해제한다. 또한, 앞서 설명한 destroy_spt 함수를 호출하여 Supplemental Page Table을 삭제한다. destroy_spt 함수는 Supplemental Page Table의 모든 엔트리를 제거하며, 메모리 누수를 방지하기 위해 관련 리소스를 정리하는 역할을 한다.

Difference from design report

기존 디자인 레포트의 구상과 유사한 방식으로 구현을 진행하였다.

Overall Limitations

아래의 결과를 보면 아직 12개의 테스트가 실패하고 있으며, 주로 Page Merge와 mmap 관련 테스트에서 오류가 발생하는 것을 확인할 수 있다. 특히, mmap-read, mmap-close, mmap-twice, mmap-write, mmap-exit 등의 테스트 케이스에서 문제가 발견되었다.

Page Merge의 경우, merge 조건을 판단하고 적절히 처리하는 로직에서 경계 조건을 제대로 다루지 못한 부분이 원인으로 보인다. 이러한 문제를 해결하기 위해 merge 로직을 재검토하고, 조건을 명확히 정의하는 추가적인 작업이 필요할 것이다.

mmap 관련 테스트에서는 매핑된 메모리의 읽기(mmap-read)와 쓰기(mmap-write), 매핑 해제(mmap-close), 프로세스 종료 시 매핑된 메모리를 정리하는 과정(mmap-exit) 등에서 실패가 발생하였다. 이는 mmap 작업 중 발생할 수 있는 다양한 상황과 경계 조건을 충분히 고려하지 못한 데서 기인한 것으로 보인다.

이러한 테스트 케이스에서의 실패는 구현 과정에서 추가적인 고려와 개선이 필요함을 보여준다.

Overall Discussion

이번 과제를 통해 Eviction Policy의 필요성과 작동 방식을 이해하게 되었다. 물리 메모리는 한정된 자원을 가지고 있으므로, 효율적인 메모리 관리가 운영체제 설계에서 매우 중요하다. 특히, 메모리가 부족할 때 어떤 페이지를 스왑 아웃해야 하는지를 결정하는 Eviction Policy는 시스템 성능에 큰 영향을 미친다. 이번 과제에서는 Clock

Algorithm을 사용하여 최근에 접근되지 않은 페이지를 선택하도록 구현하였으며, 이를 통해 페이지 교체 과정이 단순한 FIFO 방식보다 효율적이라는 점을 배울 수 있었다. 이를 구현하면서 페이지 접근 여부를 추적하는 메커니즘과 물리 메모리를 효율적으로 사용하는 방법에 대해 구체적으로 이해할 수 있었다.

Result

```
pass tests/vm/pt-grow-stk-sc
FAIL tests/vm/page-linear
pass tests/vm/page-parallel
FAIL tests/vm/page-merge-seq
FAIL tests/vm/page-merge-par
FAIL tests/vm/page-merge-stk
FAIL tests/vm/page-merge-mm
pass tests/vm/page-shuffle
FAIL tests/vm/mmap-read
FAIL tests/vm/mmap-close
pass tests/vm/mmap-unmap
pass tests/vm/mmap-overlap
FAIL tests/vm/mmap-twice
FAIL tests/vm/mmap-write
FAIL tests/vm/mmap-exit
FAIL tests/vm/mmap-shuffle
pass tests/vm/mmap-bad-fd
FAIL tests/vm/mmap-clean
FAIL tests/vm/mmap-inherit
pass tests/vm/mmap-misalign
pass tests/vm/mmap-null
pass tests/vm/mmap-over-code
pass tests/vm/mmap-over-data
pass tests/vm/mmap-over-stk
FAIL tests/vm/mmap-remove
```